

[Edit](#)

Refactor and Optimize Code with GitHub Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo, a company that wants to improve its development workflow and code quality. In this Challenge Lab, you will refactor and optimize code with GitHub Copilot. First, you will set up your Visual Studio Code environment with the necessary GitHub Copilot extensions and generate initial Python code. Next, you will leverage Copilot Chat to refactor this code for improved efficiency and clarity. Then, you will enhance the code quality further by automatically generating comprehensive documentation and unit tests. Finally, you will commit the finalized, refactored code to a new remote repository on GitHub.

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

📌 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Set up the development environment and generate initial code

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, Password, and Account URL into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

GitHub Account URL

- Authenticate **GitHub Copilot** with your GitHub account by signing in as **<GithubUN>** by using **<GithubPW>** as the password in **Visual Studio Code**.



Expand this hint for guidance on authenticating the extensions with your GitHub account. ^

- On the Windows taskbar, in Search, enter **T** Visual Studio Code, and then select the app to open it.
- In the bottom right-hand corner, select *Signed out*, select **Sign in to use Copilot**, select **Sign In**, and then sign in as **T** <GithubUN> by using **T** <GithubPW> as the password.
- Select **Continue**, and then select **Open** in the pop-up window.
- Return to **Visual Studio Code** and verify the successful sign-in status in the bottom status bar.



Want to learn more? Review the documentation on [Configuring GitHub Copilot in your environment](#).



If the sign-in notification does not appear automatically: ^

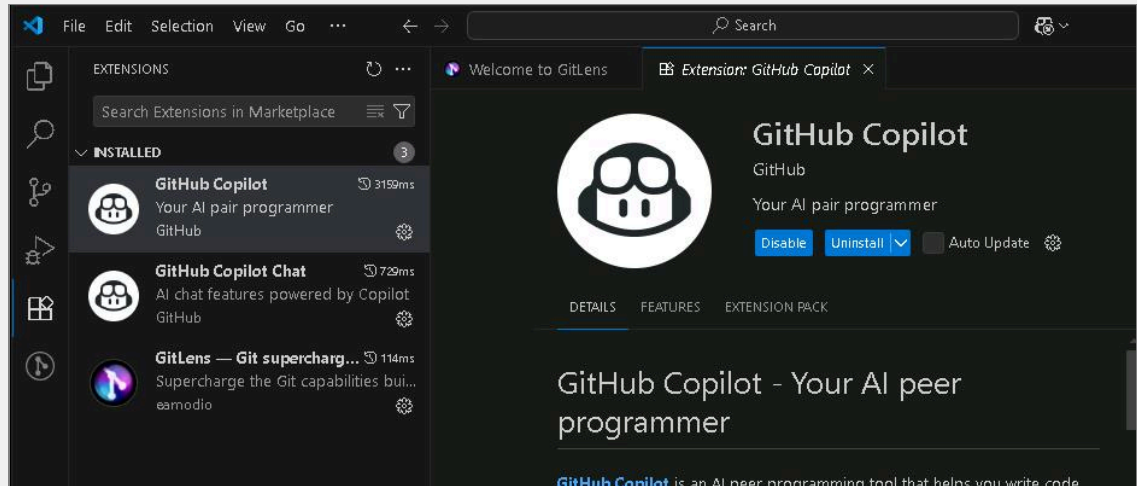
You can manually trigger the authentication by selecting the **Accounts** icon at the bottom of the Activity Bar, finding the **GitHub** section, and selecting **Sign In**.

- If necessary, install the **GitHub Copilot** and **GitHub Copilot Chat** extensions in **Visual Studio Code**.

💡 Expand this hint for guidance on installing the GitHub Copilot and GitHub Copilot Chat extensions. ^

- In **Visual Studio Code**, select the **Extensions** icon in the Activity Bar.
- In the search bar, enter **T** GitHub Copilot.
- From the results, find the **GitHub Copilot** extension by GitHub, and select **Install**.

Note: If **Update to** appears, then the extension is already installed.



- In the search bar, enter **T** GitHub Copilot Chat.
- From the results, find the **GitHub Copilot Chat** extension by GitHub, and select **Install**.

Note: If **Update to** appears, then the extension is already installed.

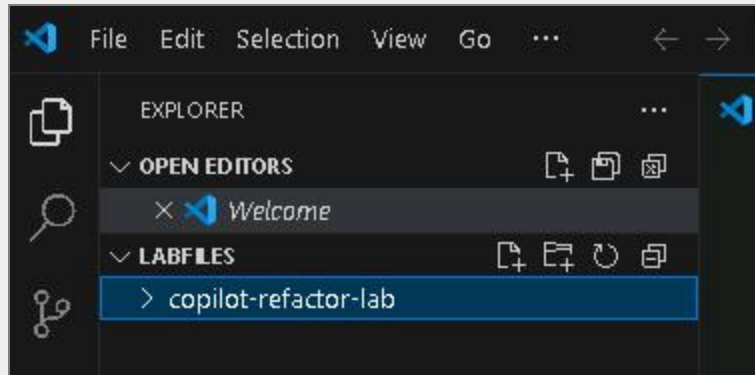
💡 Want to learn more? Review the documentation on [getting started with GitHub Copilot](#).

📄 The **Visual Studio Code** Marketplace contains thousands of extensions. Always ensure you are installing the one published by the correct, trusted author (in this case, **GitHub**).

- Create a new directory named **T** copilot-refactor-lab in the **T** D:\LabFiles directory.

💡 Expand this hint for guidance on creating a directory.

- In **VS Code**, select the Explorer icon, select *Open Folder*, navigate to the `T D:\LabFiles` folder, and then select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- In the Explorer panel, select the *LABFILES* folder, select the **New Folder** icon, and then enter `T copilot-refactor-lab`.

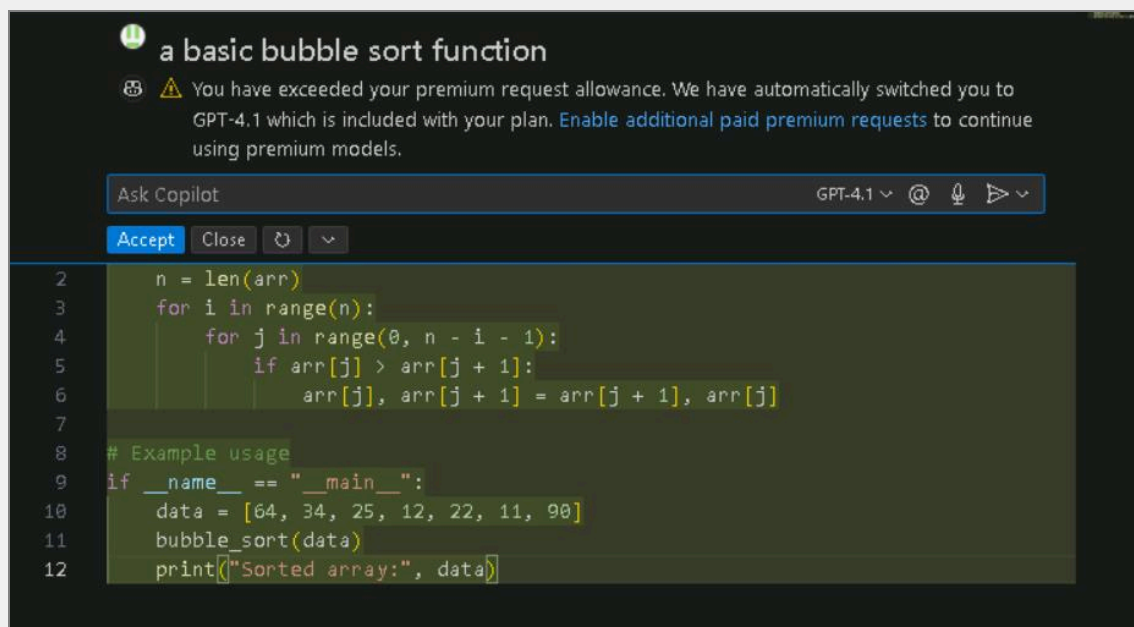


- Create a Python file named `T main.py` in the **copilot-refactor-lab** directory, and generate a bubble sort function by using **Copilot Inline Chat** and the prompt `T # a basic bubble sort function`.

💡 Expand this hint for guidance on creating a Python file named `main.py` and generating a bubble sort function. ^

- In **VS Code** Explorer, select the **copilot-refactor-lab** folder, and then select **New File**.
- Name the file `T main.py`, and press **Enter**.
- In the **main.py** file, select **Open chat** to open **Copilot Inline Chat**, and then enter `T # a basic bubble sort function`.
- Press **Enter**, and wait for GitHub Copilot to suggest the code for the function.

Note: Copilot's suggestions will vary, and your results may look similar to the screenshot below.



```
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7
8 # Example usage
9 if __name__ == "__main__":
10     data = [64, 34, 25, 12, 22, 11, 90]
11     bubble_sort(data)
12     print("Sorted array:", data)
```

- Select **Accept** to accept the suggestion.

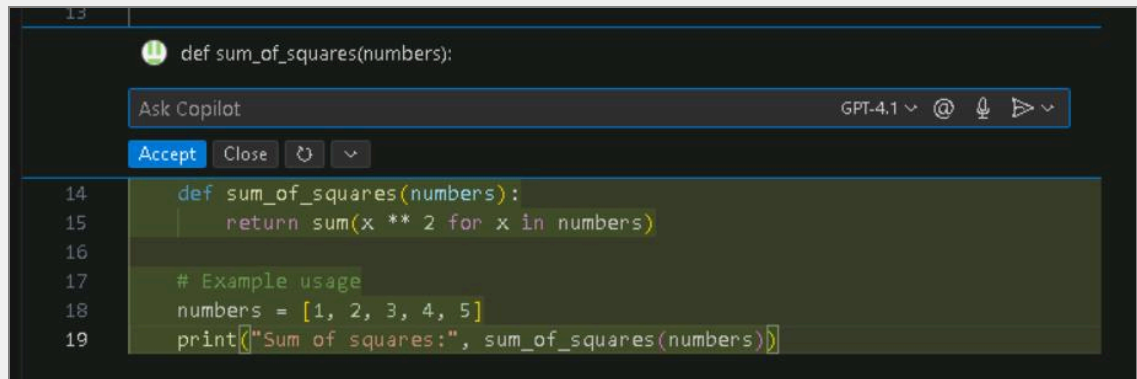
💡 Want to learn more? Review the documentation on [getting started with GitHub Copilot Chat in VS Code](#).

📄 GitHub Copilot provides suggestions based on the context of the code you are writing, including comments. A well-written comment often leads to a more accurate code suggestion.

- Add a `T sum_of_squares` function by using inline code completion and the prompt `T def sum_of_squares(numbers)`.

💡 Expand this hint for guidance on adding a `sum_of_squares` function using inline code completion. ^

- On a new line after the first function, in the **Copilot Inline Chat** editor, enter `T` `def sum_of_squares(numbers)`.
- Wait for GitHub Copilot to provide an inline suggestion for the body of the function.



```
13
def sum_of_squares(numbers):
    Ask Copilot GPT-4.1 @
    Accept Close
14 def sum_of_squares(numbers):
15     return sum(x ** 2 for x in numbers)
16
17 # Example usage
18 numbers = [1, 2, 3, 4, 5]
19 print("Sum of squares:", sum_of_squares(numbers))
```

- Select **Tab** to accept the inline suggestion.

⚠️ Inline suggestions may take a moment to appear. If a suggestion does not show up immediately, try pressing **Enter** to give Copilot more context.

💡 Want to learn more? Review the documentation on [using GitHub Copilot code suggestions](#).

- Save the **main.py** file by pressing **Ctrl+S**.

⚠️ If asked to Install *Python*, select **Accept**.

After the install, close the *Welcome* tab and return to the **main.py** file by selecting it from the tabs at the top.

Check your work

Verify

Refactor existing code for clarity and efficiency

Hints Enabled

No Yes

- Request a more efficient sorting function from **Copilot Chat** by using the following prompt: T
@workspace /explain how I can refactor the bubble_sort function to be more efficient?.

💡 Expand this hint for guidance on requesting a more efficient sorting function from Copilot Chat. ^

- In the **main.py** file, locate the `bubble_sort` function and place the cursor in the function.
- In the **Copilot Inline Chat** editor (**Ctrl+I**), enter the prompt provided above.

```
16
17     # Example usage
18     numbers = [1, 2, 3, 4, 5]
19     print("Sum of squares:", sum_of_squares(numbers))
20
21
```

@workspace /explain how can I refactor the bubble_sort function to be more efficient? GPT-4.1 @ 🔊 ▶

- Select **Send**.
- Review the explanation and the more efficient code provided by Copilot, and then in the right-hand side of the new code window, select **Apply in editor**.

```
main.py
1 def bubble_sort(arr):
```

 /explain X @workspace /explain how can I refactor the bubble_sort function to be more efficient?

 You can make `bubble_sort` more efficient by adding a flag to detect if any swaps occurred during a pass. If no swaps happen, the array is already sorted and you can exit early. This avoids unnecessary passes over an already sorted array.

Here's how you can refactor it:

```
# ...existing code...
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Track if any swaps happen
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break # Exit early if no swaps
# ...existing code...
```

Why this helps:
If the array becomes sorted before all passes are complete, the function stops early, improving performance for nearly-sorted arrays. This is a common optimization for bubble sort.

Ask Copilot GPT-4.1 @ 🔊 ▶

Close View in Chat ↻

The screenshot shows the VS Code interface with a file named `main.py`. A Copilot refactor suggestion is displayed for the `bubble_sort` function. The suggestion includes:

- Key gotchas:**
 - These reduce comparisons but don't change worst-case complexity.
 - Keep typing imports: from typing import Iterable, List
 - For large inputs use `sorted()`.
- Improved function (replace your current bubble_sort with one of these):**

```
# ...existing code...
def bubble_sort(iterable: Iterable[int]) -> List[int]:
    """Return a new list sorted in ascending order using an optimized bubble sort.

    This version tracks the last swap position and reduces the inner-loop range
    accordingly, which avoids comparing already-sorted tails.
    """
    a = list(iterable)
    n = len(a)
    # While the unsorted prefix length > 1
    ...
```

Below the suggestion, the full code for the `bubble_sort` function is shown, including the inner loop logic for swapping elements.

- If a pop-up appears at the top of **VS Code**, Select where to apply the code block, choose **Active editor 'copilot-refactor-lab\main.py'**.
- Select **Keep** to accept the changes.



Want to learn more? Review the documentation on [using GitHub Copilot Chat in VS Code](#).



The **@workspace** command tells Copilot to consider the context of all files in your current **VS Code** workspace.

This allows it to provide more relevant and accurate answers based on your entire project, not just the currently active file.

- Add a comment to the **bubble_sort** function by using the following **Copilot Inline Chat** prompt:



Add a comment to the `bubble_sort` function stating that it has been refactored.

- Simplify the `sum_of_squares` function by using the `simplify this` prompt in the inline chat.

💡 Expand this hint for guidance on simplifying the `sum_of_squares` function using inline chat. ^

- In the **main.py** file, highlight the line of code inside the `sum_of_squares` function that performs the calculation.

```
print("Sorted array:", data)

def sum_of_squares(numbers):
    return sum(x ** 2 for x in numbers)

# Example usage
```

- In the inline chat box, enter `/T simplify this`, and then press **Enter**.

```
18 print("Sorted array:", data)
19
20 def sum_of_squares(numbers):
21     return sum(x ** 2 for x in numbers)
22
23 # Example usage
```

- Review the suggestion provided by Copilot, and then select **Accept**.

📄 The inline chat feature allows you to have a conversation with Copilot directly within your code editor, making it easy to refactor or understand specific code snippets without switching context.

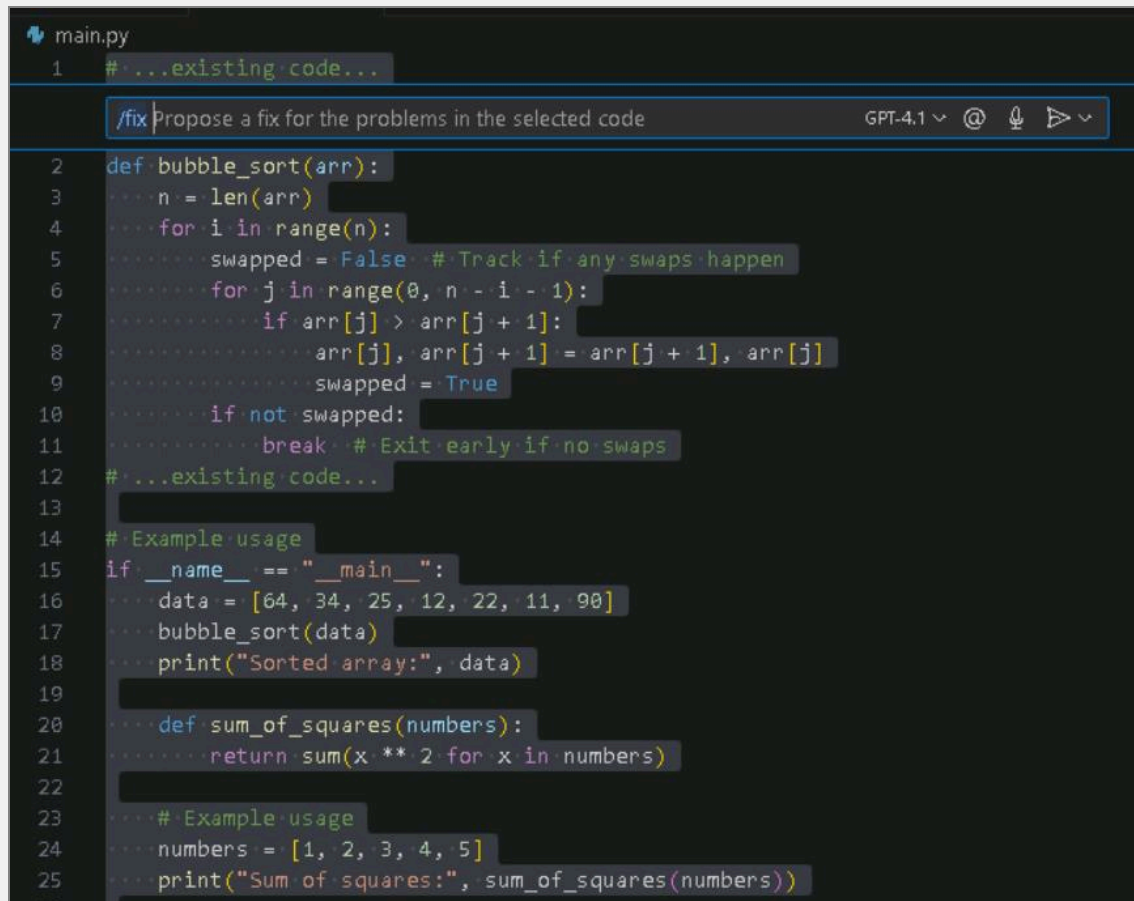
- Add a comment to the **sum_of_squares** function by using the following **Copilot Inline Chat** prompt:

📄 Add a comment to the `sum_of_squares` function stating that it has been simplified.

- Correct potential bugs in the sorting function with the `/T /fix` command.

💡 Expand this hint for guidance on correcting potential bugs in the sorting function with the `/fix` command. ^

- In the **main.py** file, highlight the entire refactored sorting function.
- In the chat input box, enter `T` `/fix` and select **Send**.

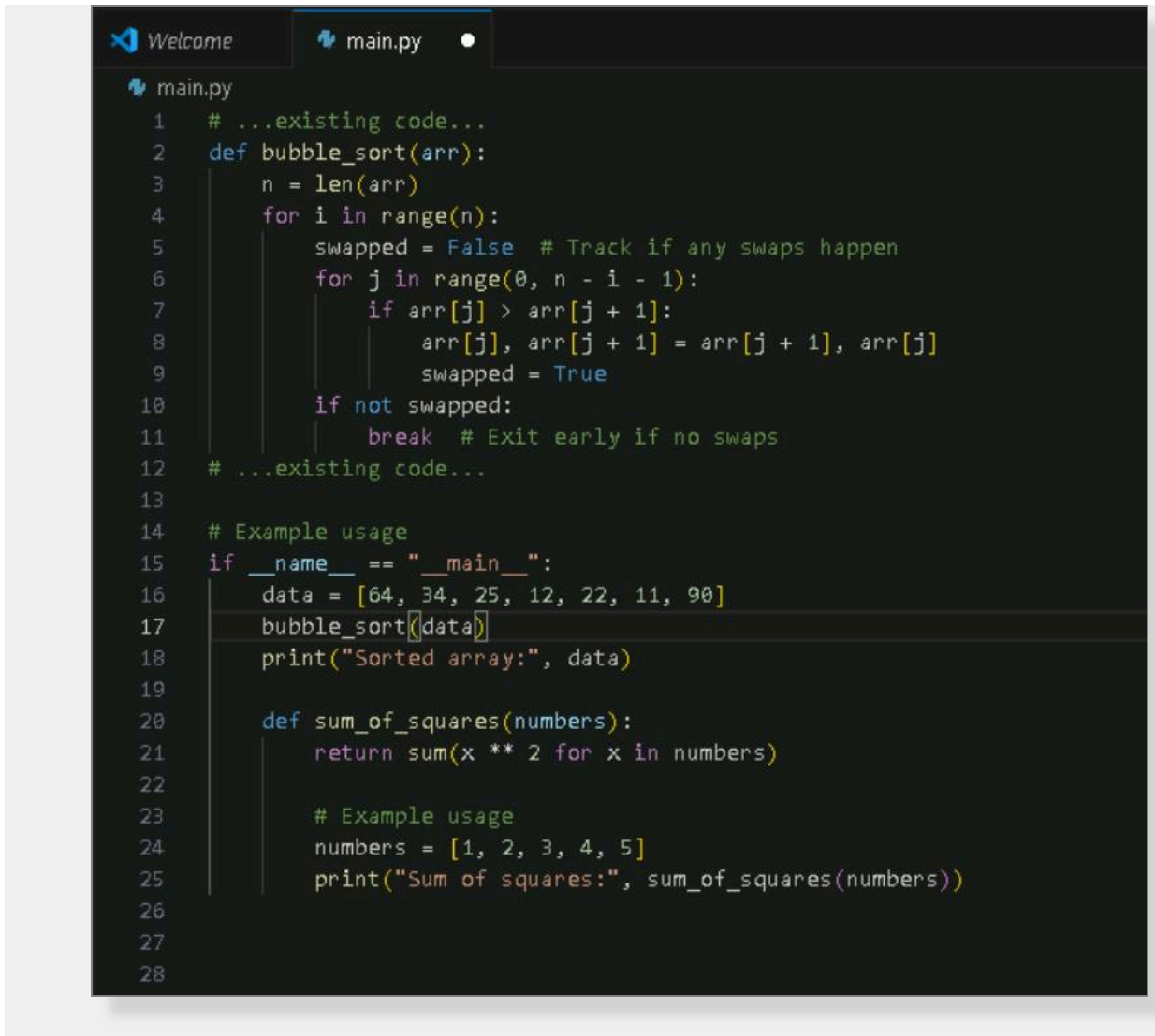


```
main.py
1  #...existing code...

2  def bubble_sort(arr):
3      n = len(arr)
4      for i in range(n):
5          swapped = False # Track if any swaps happen
6          for j in range(0, n - i - 1):
7              if arr[j] > arr[j + 1]:
8                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
9                  swapped = True
10             if not swapped:
11                 break # Exit early if no swaps
12 #...existing code...
13
14 # Example usage
15 if __name__ == "__main__":
16     data = [64, 34, 25, 12, 22, 11, 90]
17     bubble_sort(data)
18     print("Sorted array:", data)
19
20 def sum_of_squares(numbers):
21     return sum(x ** 2 for x in numbers)
22
23 # Example usage
24 numbers = [1, 2, 3, 4, 5]
25 print("Sum of squares:", sum_of_squares(numbers))
```

- Copilot will analyze the selected code. Review any suggestions and select **Accept** if changes are proposed.

Note: Copilot's suggestions will vary, and your results may look similar to the screenshot below.



```
1 # ...existing code...
2 def bubble_sort(arr):
3     n = len(arr)
4     for i in range(n):
5         swapped = False # Track if any swaps happen
6         for j in range(0, n - i - 1):
7             if arr[j] > arr[j + 1]:
8                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
9                 swapped = True
10        if not swapped:
11            break # Exit early if no swaps
12 # ...existing code...
13
14 # Example usage
15 if __name__ == "__main__":
16     data = [64, 34, 25, 12, 22, 11, 90]
17     bubble_sort(data)
18     print("Sorted array:", data)
19
20     def sum_of_squares(numbers):
21         return sum(x ** 2 for x in numbers)
22
23     # Example usage
24     numbers = [1, 2, 3, 4, 5]
25     print("Sum of squares:", sum_of_squares(numbers))
26
27
28
```

 Want to learn more? Review the documentation on [using slash commands in Copilot Chat](#).

- Save the **main.py** file by pressing **Ctrl+S**.

 If the code is already correct, Copilot's **/fix** command might not suggest any changes. This command is most useful for identifying subtle errors or logical flaws in your code.

Check your work

Verify

Enhance code quality with documentation and tests

Hints Enabled

No Yes

- Generate docstrings for all functions by using the `/doc` command in the **GitHub Copilot Chat** window.

 Expand this hint for guidance on generating docstrings for all functions with the `/doc` command. 

- In the **main.py** file, press **Ctrl+Alt+I** to open the **GitHub Copilot Chat** window.
- In the **Copilot Chat** window, next to *Add Context...*, select the **main.py** file.
- In the chat input box, enter `/doc`, and then select **Send**.
- Review the generated documentation, select **Apply in editor**.
- If a pop-up appears at the top of **VS Code**, *Select where to apply the code block*, choose **Active editor 'copilot-refactor-lab\main.py'**.
- Select **Keep** to insert the docstrings into your code.

 A *docstring* is a string literal that occurs as the first statement in a module, function, class, or method definition. 

It is used to document what the code does, its parameters, and what it returns, making the code easier for other developers to understand and use.

- Save the **main.py** file by pressing **Ctrl+S**.
- In the **copilot-refactor-lab** project directory, install **pytest** from the terminal by running `python -m pip install pytest`.

💡 Expand this hint for guidance on installing pytest.

- In **VS Code**, select **Terminal** from the menu, and then select **New Terminal**.
- In the **VS Code** Terminal, run the following command to move to the project directory:

```
cd .\copilot-refactor-lab\
```

- Run the following command to install pytest:

```
python -m pip install pytest
```

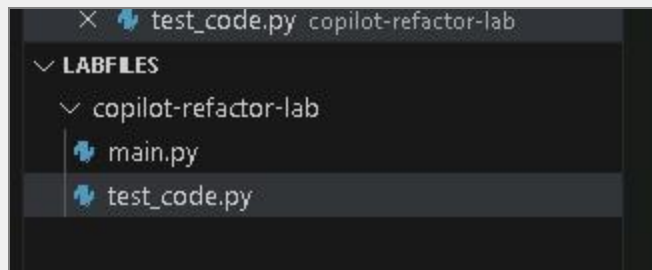
- Close the pop-up that says *Package installed globally. We recommend using a virtual environment for Python projects.* by selecting the **x** at the top.
- Wait for the installation to complete in the terminal.

⚠ You can safely ignore and close the pop-up that says *Package installed globally. We recommend using a virtual environment for Python projects.*

- Create a new test file named **T** test_code.py.

💡 Expand this hint for guidance on creating a new test file named test_code.py.

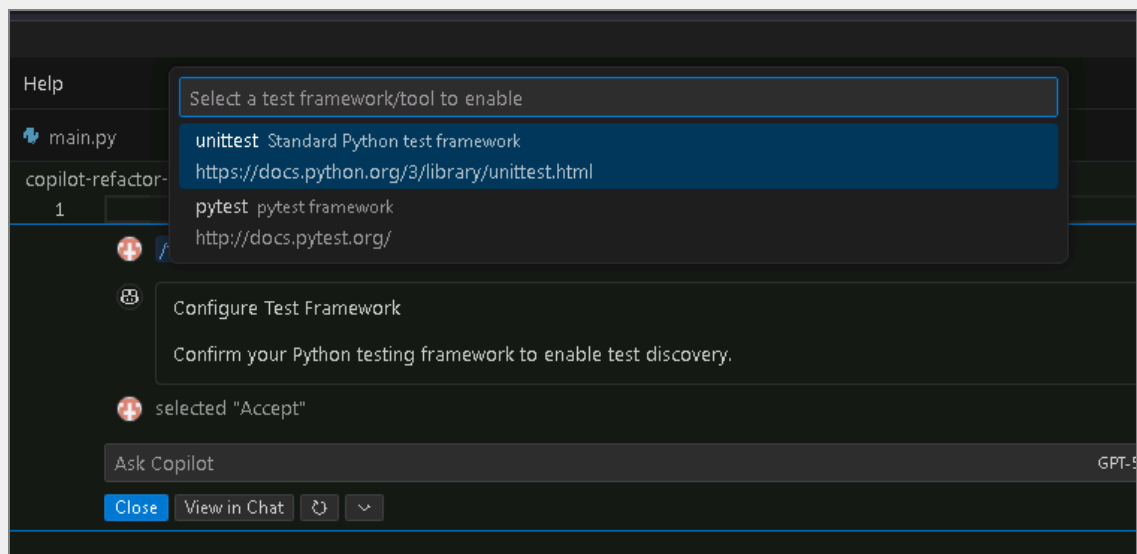
- In the Explorer panel, in the **copilot-refactor-lab** folder, select **New File**.
- Enter the file name **T** test_code.py and ensure it is saved in the same directory as **main.py**.



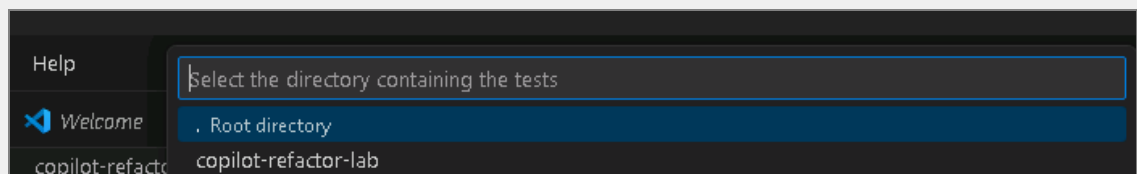
- Set up the testing environment by using the **T** @workspace /setupTests command in **Copilot Chat**.

💡 Expand this hint for guidance on setting up the testing environment.

- In the **test_code.py** file, press **Ctrl+Alt+I** to open the **GitHub Copilot Chat** window, if it is not already open.
- In the Copilot Chat window, select *Add Context...*, and then select the **test_code.py** and **main.py** files.
- In the Copilot Chat window, enter the prompt provided above, and select **Send**.
- In the Copilot Chat window, select **pytest** for the testing framework.
- In the Copilot Chat window, select **Configure Test Framework**.
- In the search box at the top of **VS Code**, select **pytest pytest framework**.



- In the search box, when asked to *Select the directory containing the tests*, select **copilot-refactor-lab**.



- Generate T **pytest** unit tests for your functions using **Copilot Chat** and the following prompt T
@workspace /tests generate unit tests for the functions in main.py using pytest.

💡 Expand this hint for guidance on generating pytest unit tests for your functions using Copilot Chat. ^

- In the **test_code.py** file, press **Ctrl+Alt+I** to open the **GitHub Copilot Chat** window, if it is not already open.
- In the Copilot Chat window, select *Add Context...*, and then select the **test_code.py** and **main.py** files.
- In the Copilot Chat window, enter the prompt provided above, and select **Send**.
- After Copilot generates the suggested test code, select **Keep**.

```

main.py  test_code.py ×
copilot-refactor-lab > test_code.py > ...
1  import pytest
2  from main import bubble_sort, sum_of_squares
3
4  def test_bubble_sort_empty_and_single():
5      assert bubble_sort([]) == []
6      assert bubble_sort(()) == []
7      assert bubble_sort([1]) == [1]
8      assert bubble_sort((42,)) == [42]
9
10 @pytest.mark.parametrize("input_iter,expected", [
11     ([3, 1, 2], [1, 2, 3]),
12     ([1, 2, 3], [1, 2, 3]),
13     ([3, 2, 1], [1, 2, 3]),
14     ([1, 1, 1], [1, 1, 1]),
15     ([5, -1, 3, -1], [-1, -1, 3, 5]),
16     ((4, 2, 4), [2, 4, 4]),
17     (range(3, -1, -1), [0, 1, 2, 3]),
18 ])
19 def test_bubble_sort_various(input_iter, expected):
20     result = bubble_sort(input_iter)
21     assert isinstance(result, list)
22     assert result == expected
23
24 def test_bubble_sort_does_not_modify_input_list():
25     data = [4, 3, 2, 1]
26     copy = data.copy()
27     _ = bubble_sort(data)
28     assert data == copy # original should remain unchanged
29

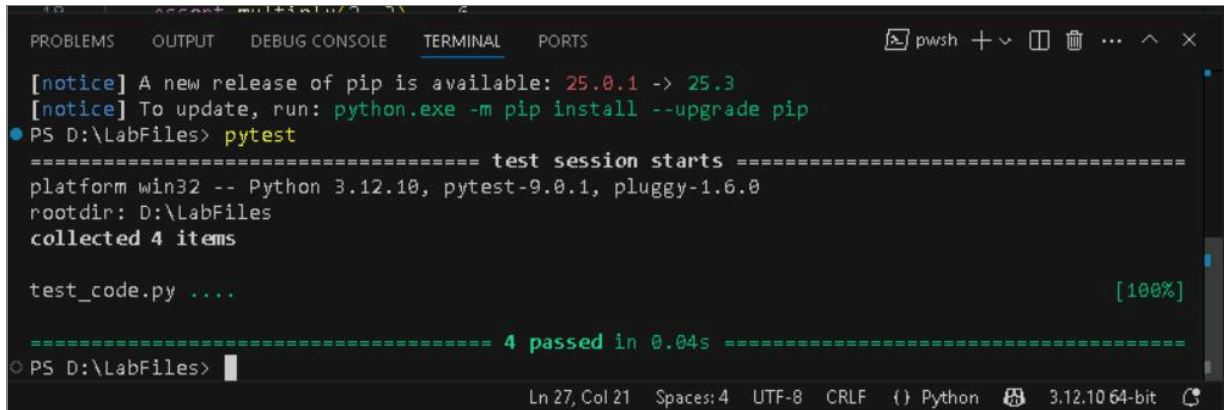
```

💡 Want to learn more? Review the documentation on [generating unit tests with Copilot Chat](#).

- Correct any indentation issues or errors by running the `T` /fix command in the **Copilot Inline Chat**.
- Save the **test_code.py** file by pressing **Ctrl+S**.

⚠ To run Python code and tests, you must have a Python interpreter installed on your system and the official **Python** extension for **Visual Studio Code** installed and enabled.

- Execute the tests by using the `T` `pytest` command, and then observe the terminal output to confirm that all tests pass.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[notice] A new release of pip is available: 25.0.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS D:\LabFiles> pytest
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.1, pluggy-1.6.0
rootdir: D:\LabFiles
collected 4 items

test_code.py .... [100%]

===== 4 passed in 0.04s =====
PS D:\LabFiles>
```

💡 Want to learn more? Review the documentation on [Pytest: How to invoke pytest](#).

📄 Ensure the terminal's current working directory is the folder containing your **main.py** and **test_code.py** files. If not, use the `T` `cd` command to navigate to the correct directory before running `T` `pytest`.

Check your work

Verify

Finalize and commit refactored code to GitHub

Hints Enabled

No Yes

- Set your Git Identity in the **VS Code** integrated terminal by using the `git config` command and using `<Name>` as the **username** and `<GithubUN>` as the **email address**.

 Expand this hint for guidance on configuring basic Git settings, such as username and email. 

- In **VS Code**, select **Terminal**, and then select **New Terminal**, if it is not already open.
- Enter `git config --global user.name "<Name>"` in the terminal to set your global Git username.
- Enter `git config --global user.email "<GithubUN>"` to set your global Git email address.

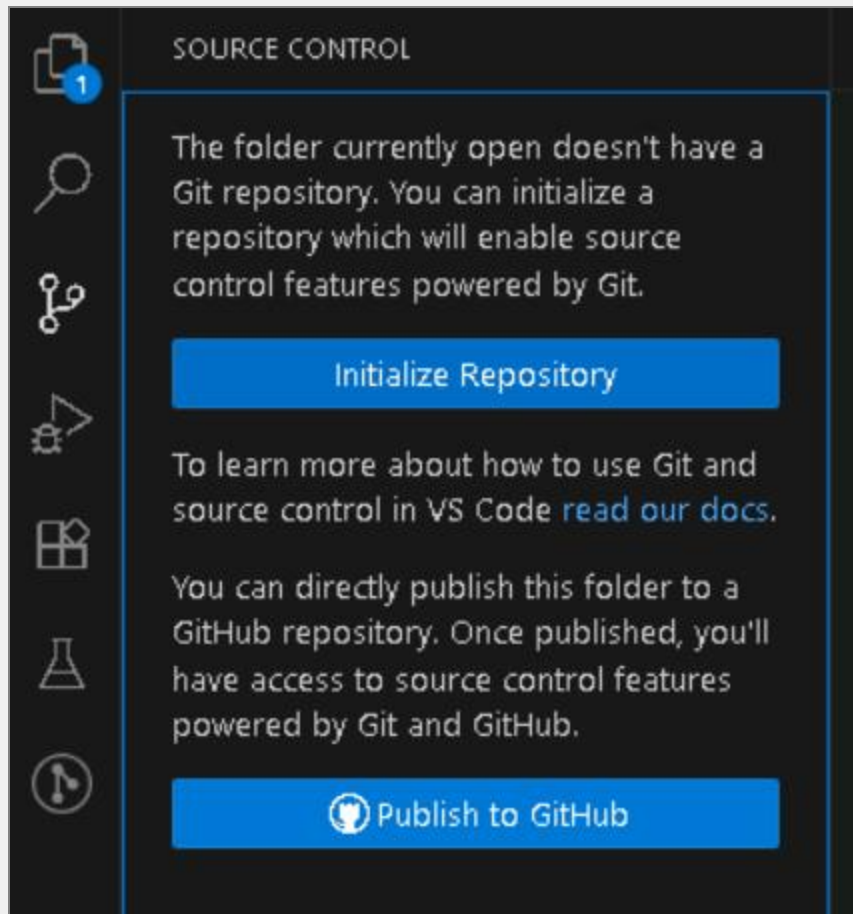
 Want to learn more? Review the documentation on [Git configuration](#).

 Use `git config --list` to verify your configurations.

- Initialize a local Git repository.

💡 Expand this hint for guidance on initializing a local Git repository.

- Select the **Source Control** icon in the Activity Bar.



- Select the **Initialize Repository** button.

💡 Want to learn more? Review the documentation on [using Version Control in VS Code](#).

- Open **Microsoft Edge**, go to <https://github.com>, and then sign in as [<GithubUN>](#) by using [<GithubPW>](#) as the password.

 Expand this hint for guidance on signing in to a GitHub account.

- On the Windows taskbar, in Search, enter `T` Edge, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter `T` <https://github.com>, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as `T` <GithubUN> using `T` <GithubPW> as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

- Create a public GitHub repository named `T` copilot-refactor-lab, and connect it as a remote repository by running `T` `git remote add origin` in the integrated terminal.

 Expand this hint for guidance on creating a public GitHub repository and connecting it as a remote repository.

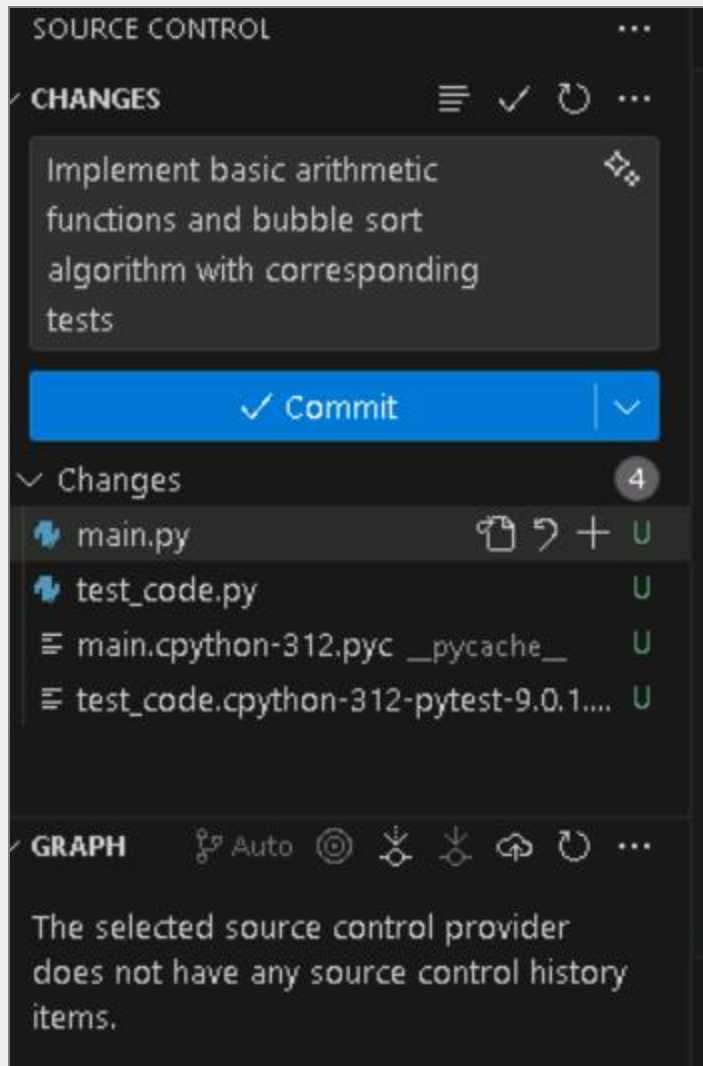
- On the GitHub homepage, select the + icon in the top-right corner, and then select **New repository**.
- For the repository name, enter `T` copilot-refactor-lab.
- Ensure the visibility is set to **Public**, and then select **Create repository**.
- From the new repository page, select the copy icon next to the *HTTPS URL*.
- Return to **Visual Studio Code**, and in the terminal, enter `T` `git remote add origin`, add a space, paste the URL you copied, and then press **Enter**.

 Want to learn more? Review the documentation on [managing remote repositories](#). If you are using HTTPS and have two-factor authentication enabled on GitHub, you may need to use a personal access token instead of your password when prompted for credentials by Git.

- Generate a commit message using Copilot.

💡 Expand this hint for guidance on generating a commit message with Copilot.

- Select the **Source Control** icon.
- Select the sparkle icon (**Generate Commit Message with Copilot**), next to the **Message** input box.



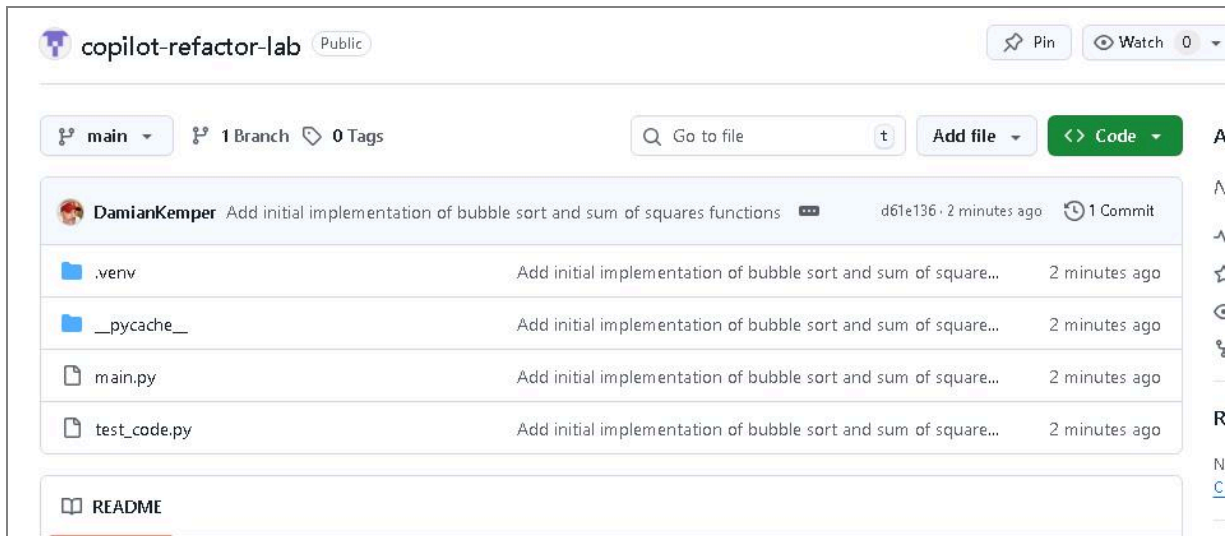
- Review the suggested commit message, and then select **Commit**.
- On the pop-up, *Would you like to stage all your changes and commit them directly?*, select **Yes**.
- Select **Publish Branch**.
- If necessary, sign in to GitHub by selecting **Sign in with your browser**.

💡 Want to learn more? Review the documentation on [generating commit messages with GitHub Copilot](#).

❓ A good commit message is concise and descriptive. It should summarize the "what" and "why" of the changes. ^

Using Copilot to generate these messages saves time and helps maintain a clean, understandable project history.

- Refresh your GitHub homepage or navigate to [T <AccountURL>/copilot-refactor-lab](https://github.com/<AccountURL>/copilot-refactor-lab) to see your public repository.



Check your work

Verify

Summary

Congratulations, you have completed the **Refactor and Optimize Code with GitHub Copilot** Challenge Lab.

You have accomplished the following:

- Set up the development environment and generated initial code.
- Refactored existing code for clarity and efficiency.
- Enhanced code quality with documentation and tests.
- Finalized and committed refactored code to GitHub.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]

- Refactor Python Code by Using Copilot [Guided]
- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

